Formative Assignment: Advanced Linear Algebra (PCA)

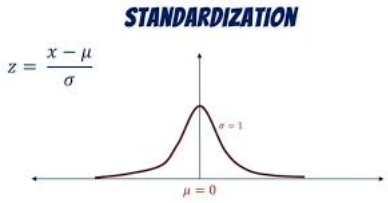
This notebook will guide you through the implementation of Principal Component Analysis (PCA). Fill in the missing code and provide the required answers in the appropriate sections. You will work with a dataset that is Africanized .

1. Make sure to display outputs for each code cell when submitting.
2. Do not write all your code on one cell
3. Do not use any libraries aside from numpy

Step 1: Load and Standardize the Data

Before applying PCA, we must standardize the dataset. Standardization ensures that all features have a mean of 0 and a standard deviation of 1, which is essential for PCA. Fill in the code to standardize the dataset.

STRICTLY - Write code that implements standardization based on the image below




```
# Step 1: Load and Standardize the data (use of numpy only allowed)
import numpy as np

raw_data = np.genfromtxt('covid_africa.csv', delimiter=',', dtype=str, encoding='utf-8', autostrip=True)

header = raw_data[0]
data = raw_data[1:]

# Extract numeric columns and handle missing values
numeric_raw = data[:, 1:].copy()
numeric_raw[numeric_raw == ''] = 'nan'
numeric_data = numeric_raw.astype(float)
col_means = np.nanmean(numeric_data, axis=0)
for i in range(numeric_data.shape[1]):
    numeric_data[np.isnan(numeric_data[:, i]), i] = col_means[i]

# Standardize: (X - mean) / std
mean = np.mean(numeric_data, axis=0)
std = np.std(numeric_data, axis=0)
standardized_data = (numeric_data - mean) / std

standardized_data[:5]  # Display the first few rows of standardized data
```

```
array([[ 0.07532886,  0.14526313, -0.04361689,  4.40494752, -0.28846506,
        -0.30718014, -0.47365043, -0.75954597,  0.51936724],
       [-0.21003692, -0.19557076, -0.18936555, -0.36847384, -0.32943247,
        -0.49289223, -0.15918165, -0.5829783 ,  0.24205879],
       [-0.34266751, -0.31758751, -0.32766592, -0.3700479 , -0.3406553 ,
        -0.57330366, -0.38111916, -0.56215057, -0.35545959],
       [ 0.17544721, -0.13583699,  0.21988802, -0.34661191,  1.48538573,
         1.59780474, -0.02854414,  3.10274358, -0.6333257 ],
       [-0.35288094, -0.30153449, -0.33910555, -0.36654999, -0.35702306,
        -0.56373087, -0.46918063, -0.73065861, -0.10514088]])
```

Step 3: Calculate the Covariance Matrix

The covariance matrix helps us understand how the features are related to each other. It is a key component in PCA.

```
# Step 3: Calculate the Covariance Matrix
cov_matrix = np.cov(standardized_data, rowvar=False)
cov_matrix
```

```
array([[ 1.01886792,  0.99563389,  0.99188965,  0.47554039,  0.11407379,
         0.51618861,  0.96751072,  0.25931321,  0.2335514 ],
```

```
       [ 0.99563389,  1.01886792,  0.96174193,  0.53569107,  0.09304069,
         0.50817262,  0.91524655,  0.20883333,  0.23333338],
       [ 0.99188965,  0.96174193,  1.01886792,  0.46145439,  0.08638073,
         0.39851269,  0.96994967,  0.23236287,  0.2474435 ],
       [ 0.47554039,  0.53569107,  0.46145439,  1.01886792, -0.04442246,
         0.10875087,  0.41170131, -0.07058693,  0.31937834],
       [ 0.11407379,  0.09304069,  0.08638073, -0.04442246,  1.01886792,
         0.66789186,  0.11230481,  0.34100352, -0.17839911],
       [ 0.51618861,  0.50817262,  0.39851269,  0.10875087,  0.66789186,
         1.01886792,  0.4050237 ,  0.62408757, -0.17574918],
       [ 0.96751072,  0.91524655,  0.96994967,  0.41170131,  0.11230481,
         0.4050237 ,  1.01886792,  0.25976387,  0.34381408],
       [ 0.25931321,  0.20883333,  0.23236287, -0.07058693,  0.34100352,
         0.62408757,  0.25976387,  1.01886792, -0.27314237],
       [ 0.2335514 ,  0.23333338,  0.2474435 ,  0.31937834, -0.17839911,
        -0.17574918,  0.34381408, -0.27314237,  1.01886792]])
```

In a paragraph that has a maximum if 5 Lines, Explain why we need to compute a covariance matrix, provide atleast 2 reasons

A covariance matrix is a square matrix that summarizes the relationships between multiple variables in a dataset.It is used to map the shape, direction, and magnitude of data spread across all variables of a dataset at once and can show relationships between 2 variables.

Step 4: Perform Eigendecomposition

Eigendecomposition of the covariance matrix will give us the eigenvalues and eigenvectors, which are essential for PCA. Fill in the code to compute the eigenvalues and eigenvectors of the covariance matrix.

```
# Step 4: Perform Eigendecomposition
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
eigenvalues, eigenvectors
```

```
(array([4.64436815, 2.11941207, 0.83697068, 0.67388384, 0.56856986,
        0.24490283, 0.00675082, 0.02395363, 0.05099943]),
 array([[ 0.45886645, -0.0565882 , -0.14014516, -0.02490356, -0.13688307,
          0.08146437,  0.85956897,  0.0393377 , -0.01410035],
        [ 0.45143868, -0.07892546, -0.10037083,  0.07573547, -0.1398117 ,
          0.22694854, -0.28634367, -0.57997441, -0.53361761],
        [ 0.44675534, -0.09650034, -0.17696339, -0.0549254 , -0.15705576,
         -0.23639033, -0.31482751,  0.71674693, -0.25174409],
        [ 0.2484436 , -0.26757774,  0.3749676 ,  0.76161549,  0.34590091,
         -0.1145516 , -0.0009588 ,  0.03604694,  0.10931268],
        [ 0.10753514,  0.4948907 ,  0.62964327, -0.06334796, -0.32826779,
         -0.44688433,  0.06782426, -0.09202971, -0.15006944],
        [ 0.2764197 ,  0.48430358,  0.2101885 ,  0.01680426,  0.04162441,
          0.71159797, -0.14737705,  0.21664164,  0.260428  ],
        [ 0.44158919, -0.09376038, -0.11346311, -0.2181898 , -0.07697047,
         -0.28467856, -0.22691756, -0.2916622 ,  0.71568328],
        [ 0.16277121,  0.48219086, -0.26917997, -0.10466397,  0.74581255,
         -0.266753  ,  0.04269968, -0.07053465, -0.15294827],
        [ 0.12191393, -0.43461775,  0.52192595, -0.58967136,  0.38177036,
          0.12074409,  0.01990598,  0.04014535, -0.11742979]]))
```

Step 5: Sort Principal Components

Sort the eigenvectors based on their corresponding eigenvalues in descending order. The higher the eigenvalue, the more important the eigenvector. Complete the code to sort the eigenvectors and print the sorted components.

How Is Explained Variance Used In PCA?'

```
# Step 5: Sort Principal Components
sorted_indices = np.argsort(eigenvalues)[::-1]  # Sort eigenvalues in descending order
sorted_eigenvectors = eigenvectors[:, sorted_indices]  # Sort eigenvectors accordingly
sorted_eigenvectors
```

```
array([[ 0.45886645, -0.0565882 , -0.14014516, -0.02490356, -0.13688307,
         0.08146437, -0.01410035,  0.0393377 ,  0.85956897],
       [ 0.45143868, -0.07892546, -0.10037083,  0.07573547, -0.1398117 ,
         0.22694854, -0.53361761, -0.57997441, -0.28634367],
       [ 0.44675534, -0.09650034, -0.17696339, -0.0549254 , -0.15705576,
        -0.23639033, -0.25174409,  0.71674693, -0.31482751],
       [ 0.2484436 , -0.26757774,  0.3749676 ,  0.76161549,  0.34590091,
        -0.1145516 ,  0.10931268,  0.03604694, -0.0009588 ],
       [ 0.10753514,  0.4948907 ,  0.62964327, -0.06334796, -0.32826779,
        -0.44688433, -0.15006944, -0.09202971,  0.06782426],
```

```
       [ 0.2764197 ,  0.48430358,  0.2101885 ,  0.01680426,  0.04162441,
         0.71159797,  0.260428  ,  0.21664164, -0.14737705],
       [ 0.44158919, -0.09376038, -0.11346311, -0.2181898 , -0.07697047,
        -0.28467856,  0.71568328, -0.2916622 , -0.22691756],
       [ 0.16277121,  0.48219086, -0.26917997, -0.10466397,  0.74581255,
        -0.266753  , -0.15294827, -0.07053465,  0.04269968],
       [ 0.12191393, -0.43461775,  0.52192595, -0.58967136,  0.38177036,
         0.12074409, -0.11742979,  0.04014535,  0.01990598]])
```

Step 6: Project Data onto Principal Components

Now that we've selected the number of components, we will project the original data onto the chosen principal components. Fill in the code to perform the projection.

```
# Step 6: Project Data onto Principal Components
num_components = 2  # Decide on the number of principal components to keep
reduced_data = standardized_data @ sorted_eigenvectors[:, :num_components]  # Project data onto the principal components
reduced_data[:5]
```

```
array([[ 0.78963441, -2.02927434],
       [-0.66815747, -0.6289371 ],
       [-1.03717278, -0.35198796],
       [ 1.0479246 ,  3.35529075],
       [-1.07376603, -0.53776246]])
```

Step 7: Output the Reduced Data

Finally, display the reduced data obtained by projecting the original dataset onto the selected principal components.

```
# Step 7: Output the Reduced Data
print(f'Reduced Data Shape: {reduced_data.shape}')  # Display reduced data shape
reduced_data[:5]  # Display the first few rows of reduced data
```

```
Reduced Data Shape: (54, 2)
array([[ 0.78963441, -2.02927434],
       [-0.66815747, -0.6289371 ],
       [-1.03717278, -0.35198796],
       [ 1.0479246 ,  3.35529075],
       [-1.07376603, -0.53776246]])
```

Step 8: Visualize Before and After PCA

Now, let's plot the original data and the data after PCA to compare the reduction in dimensions visually.

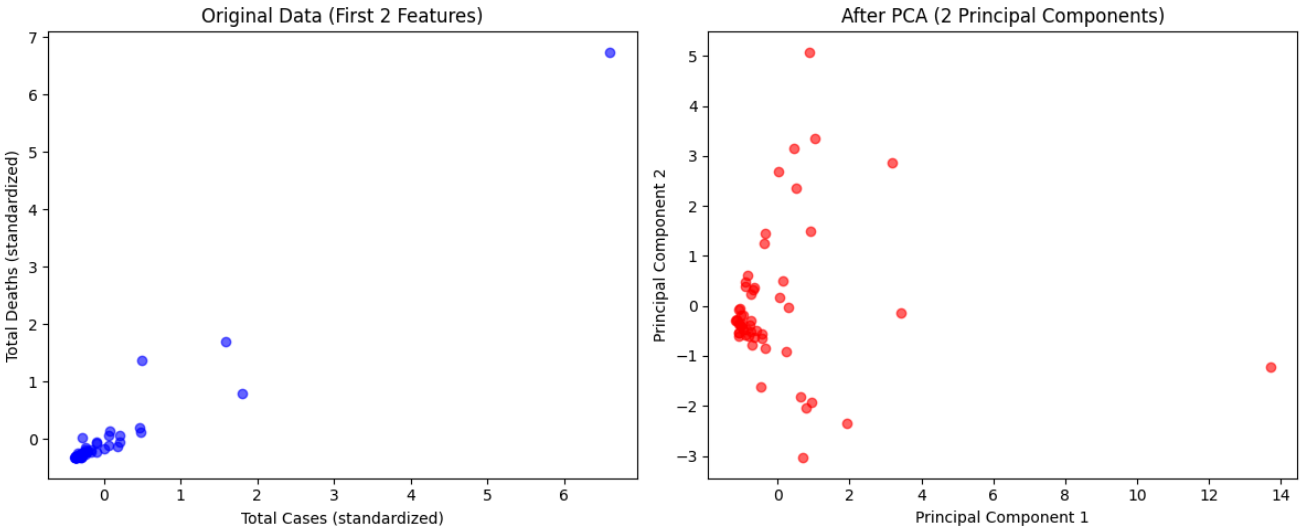
```
# Step 8: Visualize Before and After PCA
import matplotlib.pyplot as plt

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# Plot original data (first two features for simplicity)
ax1.scatter(standardized_data[:, 0], standardized_data[:, 1], color='blue', alpha=0.6)
ax1.set_title('Original Data (First 2 Features)')
ax1.set_xlabel('Total Cases (standardized)')
ax1.set_ylabel('Total Deaths (standardized)')

# Plot reduced data after PCA
ax2.scatter(reduced_data[:, 0], reduced_data[:, 1], color='red', alpha=0.6)
ax2.set_title('After PCA (2 Principal Components)')
ax2.set_xlabel('Principal Component 1')
ax2.set_ylabel('Principal Component 2')

plt.tight_layout()
plt.show()
```



Please make sure you do not use more than 5 lines to answer each of the following points:

1. Interpret the Visual you just created of the before and after PCA
2. Explain why you selected the number of principle components, more especially explain what tradeoffs you are making
3. Clearly mention based on your use case/ dataset ("economic activity", "population pressure"), What information is lost when reducing dimensions?

1. The original plot is only able to represent 2 columns here being Total deaths and Total Cases which only represent a fraction of the dataset and will need nultiple graphs to give a full picture but the "After PCA" combines all 9 numeric columns into 2 components which carry 75% of the data in the dataset with the Y-axis tells us the impact of covid per capita and X-axis the number of cases.
2. We selected 2 principle components as from the eigenvalues following Kaiser rule to pick values greater than 1, also 2 values are easier to Visualize on a plot and the 2 values account for 75% of the variance captured.
3. We lost granular data like population difference between countries and its relation to covid cases, the testing differences between countries, total cases vs how much recovered.

Please ensure this tracking sheet is completed as a group, as it will be used to monitor your individual participation within the team.

Team Members	Role/ Responsibilty	Email				
Aurele Karega	Data loading, standardization & covariance matrix	a.karega@alustudent.com				
Kelvin Ebubechukwu Aaron-Onuigbo	Eigendecomposition, PCA projection & visualization	k.aaron-onu@alustudent.com				

Task Allocation and Tracker

	Task Allocated	Assigned Member	Deadline	Completion Status (Not Started/In Progress/Completed/Did not participate)	Reviewed by Team? (Yes/No)	Comments
	Find African dataset (covid_africa.csv) with missing values & non-numeric column	Both	8/6/2026	Completed	Yes	
	Load & standardize data (numpy only)	Aurele Karega	8/6/2026	Completed	Yes	
	Calculate covariance matrix	Aurele Karega	8/6/2026	Completed	Yes	
	Eigendecomposition (eigenvalues & eigenvectors) and sorting	Kelvin Ebubechukwu Aaron-Onuigbo	9/6/2026	Completed	Yes	
	Project data onto Principal Components	Kelvin Ebubechukwu Aaron-Onuigbo	9/6/2026	Completed	Yes	
	Visualize before and after PCA	Kelvin Ebubechukwu Aaron-Onuigbo	9/6/2026	Completed	Yes	
	Interpret the data	Both	10/6/2026	Completed	Yes	

Meeting Attendance Log

Meeting Date	Agenda	Facilitator	Attendees	Key Discussion Points	Next Steps	
8/6/2026	Dataset selection & task split	Aurele Karega	Aurele Karega, Kelvin Ebubechukwu Aaron-Onuigbo	Agreed on covid_africa.csv; divided Steps 1–4 vs 5–8	Each member starts their assigned steps	
10/6/2026	Review & merge notebook	Kelvin Ebubechukwu Aaron-Onuigbo	Aurele Karega, Kelvin Ebubechukwu Aaron-Onuigbo	Reviewed outputs, checked eigenvalues, finalized written answers	Push to GitHub and submit	